Aprendizaje de Máquina
Sesión 23: Aprendizaje de máquina para textos

Angel Rodríguez

Examen sorpresa

Hoy

Propuestas

1. Martes 3 de Noviembre
2. Jueves 5 de Noviembre

El método bootstrap

El **bootstrap** consiste en obtener repetidamente muestras aleatorias **con reemplazo** a partir de una muestra original.

Supongamos que tenemos:

$$x = \{x_1, x_2, x_3, x_4, x_5\}.$$

Una posible muestra bootstrap es:

$$x^{(1)} = \{x_2, x_5, x_2, x_1, x_4\}.$$

Otra posible muestra es:

$$x^{(2)} = \{x_3, x_3, x_1, x_5, x_5\}.$$

- Cada muestra tiene tamaño $n = 5$.
- Un elemento puede aparecer varias veces.
- Algunos elementos pueden no aparecer.

Ejemplo: dos clasificadores y 10 documentos

Consideremos un conjunto de prueba con $n = 10$ documentos.
Cada documento puede pertenecer a uno de cuatro casos:

Caso	A	B
1	Correcto	Correcto
2	Correcto	Incorrecto
3	Incorrecto	Correcto
4	Incorrecto	Incorrecto

Ejemplo

	1	2	3	4	5	6	7	8	9	10	A%	B%	$\delta()$
x	AB	AB	AB	AB	AB	AB	AB	AB	AB	AB	.70	.50	.20
$x^{(1)}$	AB	AB	AB	AB	AB	AB	AB	AB	AB	AB	.60	.60	.00
$x^{(2)}$	AB	AB	AB	AB	AB	AB	AB	AB	AB	AB	.60	.70	-.10
...													
$x^{(b)}$													

$$\text{Accuracy}(A, x) = 0{,}70,$$

$$\text{Accuracy}(B, x) = 0{,}50.$$

Entonces:

$$\boxed{\delta(x) = 0{,}70 - 0{,}50 = 0{,}20.}$$

El modelo A supera a B por 20 puntos porcentuales en este test set.

Construcción de los test sets virtuales

Partimos de un conjunto original x con $n = 10$ documentos.

Para construir $x^{(1)}$:

1. Seleccionamos aleatoriamente un documento de x.
2. Copiamos su etiqueta y los resultados de ambos modelos.
3. Devolvemos el documento al conjunto.
4. Repetimos hasta obtener 10 selecciones.

Repetimos todo el proceso b veces:

$$x^{(1)}, x^{(2)}, \ldots, x^{(b)}.$$

Para cada muestra calculamos:

$$\delta(x^{(i)}) = M(A, x^{(i)}) - M(B, x^{(i)}).$$

Número de repeticiones

Se sugiere utilizar un número grande de muestras, por ejemplo $b = 10^5$.

¿Por qué debemos ajustar la diferencia?

Recordemos que observamos:

$$\delta(x) = 0{,}20.$$

Bajo H_0, queremos evaluar diferencias que serían sorprendentes si A no fuera mejor que B.

Lo natural sería contar cuantas veces $\delta(x^{(i)}) \geq 0$:

$$p = \frac{1}{b} \sum_{i=1}^{b} \mathbb{I}\left(\delta(x^{(i)}) - \delta(x) \geq 0\right).$$

Pero las muestras bootstrap se construyen a partir del conjunto original, donde A ya tiene una ventaja de 0,20.
Por ello, la distribución bootstrap de $\delta(x^{(i)})$ está aproximadamente centrada alrededor de $\delta(x)$, no alrededor de cero.

Consecuencia

No debemos tratar directamente la distribución bootstrap original como si fuera una distribución generada bajo H_0.

Necesitamos considerar las diferencias respecto de la ventaja observada.

Deducción del p-value bootstrap

La diferencia observada es $\delta(x)$.

Para cada muestra bootstrap calculamos la desviación respecto de esa diferencia:

$$\delta(x^{(i)}) - \delta(x).$$

Contamos cuándo esa desviación es al menos tan grande como $\delta(x)$:

$$\delta(x^{(i)}) - \delta(x) \geq \delta(x).$$

Sumando $\delta(x)$ en ambos lados:

$$\boxed{\delta(x^{(i)}) \geq 2\delta(x).}$$

Por tanto, el p-value empírico es:

$$\boxed{p = \frac{1}{b} \sum_{i=1}^{b} \mathbb{I}\left(\delta(x^{(i)}) \geq 2\delta(x)\right).}$$

Aquí $\mathbb{I}(\cdot)$ vale 1 si la condición se cumple y 0 en otro caso.

Práctica #8: Clasificación y significancia estadística

Objetivo: Comparar dos representaciones de texto para clasificación multiclase y determinar si existe evidencia estadística de una diferencia de rendimiento.

Modelos:

- Modelo A: `TfidfVectorizer` + `LogisticRegression`.
- Modelo B: `CountVectorizer` + `LogisticRegression`.

Procedimiento:

1. Preparar los datos y construir los pipelines.
2. Evaluar mediante 5-fold cross-validation.
3. Comparar métricas multiclase.
4. Seleccionar hiperparámetros con GridSearchCV.
5. Evaluar ambos modelos en el test set.
6. Analizar errores de clasificación.
7. Aplicar el paired bootstrap test.

Pregunta central

¿Una diferencia observada de F_1 macro entre dos clasificadores representa evidencia suficiente para concluir que uno es mejor?

Paso 1: Cargar los datos

Utilizaremos cuatro categorías del conjunto *20 Newsgroups*.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import fetch_20newsgroups
from sklearn.model_selection import train_test_split

categorias = ["sci.space", "rec.sport.baseball", "comp.graphics", "talk.
    politics.mideast"]

datos = fetch_20newsgroups(subset="train", categories=categorias, remove
    =("headers", "footers", "quotes"), shuffle=True, random_state=42)

X = np.array(datos.data, dtype=object)
y = np.array(datos.target)

print("Documentos:", len(X))
print("Categorias:", datos.target_names)
```

Paso 2: Separar entrenamiento y prueba

Reservamos el 20 % de los datos para evaluar el desempeño final.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    stratify=y,
    random_state=42
)

print("Entrenamiento:", len(X_train))
print("Prueba:", len(X_test))
```

¿Dónde está el conjunto de validación?

Utilizaremos **5-fold cross-validation** dentro del conjunto de entrenamiento.
En cada iteración:

- 4 folds se utilizan para entrenar.
- 1 fold se utiliza para validar.

El test set permanece separado durante la selección de modelos e hiperparámetros.

Paso 3: Construir los pipelines

Comparamos dos representaciones manteniendo el mismo algoritmo de clasificación.

```
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import (
    CountVectorizer,
    TfidfVectorizer
)
from sklearn.linear_model import LogisticRegression

pipeline_count = Pipeline([
    ("vectorizer", CountVectorizer()),
    ("classifier", LogisticRegression(max_iter=2000))
])

pipeline_tfidf = Pipeline([
    ("vectorizer", TfidfVectorizer()),
    ("classifier", LogisticRegression(max_iter=2000))
])

modelos = {
    "CountVectorizer": pipeline_count,
    "TfidfVectorizer": pipeline_tfidf
}
```

Paso 4: Evaluación con 5-fold cross-validation

Evaluamos ambos pipelines utilizando las mismas particiones de validación cruzada.

```
from sklearn.model_selection import StratifiedKFold, cross_validate

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

metricas = {"Accuracy": "accuracy", "Precision macro": "precision_macro", "Recall macro": "
      recall_macro", "F1 macro": "f1_macro", "F1 micro": "f1_micro"}

resultados = []

for nombre, modelo in modelos.items():
    scores = cross_validate(modelo, X_train, y_train, cv=cv, scoring=metricas, n_jobs=-1)

    fila = {"Modelo": nombre}

    for metrica in metricas:
        valores = scores["test_" + metrica]
        fila[metrica] = valores.mean()
        fila[metrica + " std"] = valores.std()

    resultados.append(fila)

df_resultados = pd.DataFrame(resultados)
print(df_resultados.round(4).to_string(index=False))
```

Paso 5: Comparar F_1 macro y micro

```
comparacion = df_resultados.set_index("Modelo")[
    ["F1 macro", "F1 micro"]
]

print(comparacion.round(4))

comparacion.plot(
    kind="bar",
    figsize=(8, 4),
    rot=0
)

plt.ylabel("F1")
plt.ylim(0, 1)
plt.title("Comparacion de F1 macro y micro")
plt.tight_layout()
plt.show()
```

Paso 6: Selección de hiperparámetros

Utilizaremos `GridSearchCV` para explorar diferentes configuraciones de cada pipeline.

```
from sklearn.model_selection import GridSearchCV

param_grid = {
    "vectorizer__ngram_range": [(1, 1), (1, 2)],
    "vectorizer__min_df": [1, 3],
    "classifier__C": [0.1, 1, 10]
}

busquedas = {}
for nombre, modelo in modelos.items():
    grid = GridSearchCV(
        estimator=modelo,
        param_grid=param_grid,
        scoring="f1_macro",
        cv=cv,
        n_jobs=-1,
        refit=True
    )

    grid.fit(X_train, y_train)
    busquedas[nombre] = grid

    print(nombre)
    print(grid.best_params_)
    print("Mejor F1 macro:", grid.best_score_)
```

Paso 7: Comparar las mejores configuraciones

Comparamos los mejores resultados de validación cruzada de ambos pipelines.

```
resumen = []

for nombre, grid in busquedas.items():
    resumen.append({
        "Modelo": nombre,
        "F1 macro CV": grid.best_score_,
        "Parametros": grid.best_params_
    })

df_resumen = pd.DataFrame(resumen)
print(df_resumen.to_string(index=False))

mejor_nombre = max(busquedas,key=lambda nombre: busquedas[nombre].best_score_)

print("Modelo seleccionado:", mejor_nombre)
```

Importante

La selección se realiza utilizando **únicamente el conjunto de entrenamiento**.
Con `refit=True`, cada mejor configuración se reentrena automáticamente con todos los datos de entrenamiento.

Conservaremos ambos modelos optimizados para realizar posteriormente el paired bootstrap test.

Paso 8: Evaluar los modelos en el test set

Obtenemos las predicciones de los dos modelos sobre exactamente los mismos documentos.

```
from sklearn.metrics import f1_score, classification_report

modelo_count = busquedas["CountVectorizer"].best_estimator_
modelo_tfidf = busquedas["TfidfVectorizer"].best_estimator_

pred_count = modelo_count.predict(X_test)
pred_tfidf = modelo_tfidf.predict(X_test)

f1_count = f1_score(y_test, pred_count, average="macro")
f1_tfidf = f1_score(y_test, pred_tfidf, average="macro")

print("F1 Count:", f1_count)
print("F1 TF-IDF:", f1_tfidf)

print(classification_report(
    y_test, pred_tfidf,
    target_names=datos.target_names,
    zero_division=0
))
```

Paso 9: Matriz de confusión

Visualizamos qué categorías se confunden con mayor frecuencia.

```
from sklearn.metrics import ConfusionMatrixDisplay

ConfusionMatrixDisplay.from_predictions(
    y_test,
    pred_tfidf,
    display_labels=datos.target_names,
    xticks_rotation=45,
    cmap="Blues"
)

plt.title("Matriz de confusion: TF-IDF")
plt.tight_layout()
plt.show()
```

Preguntas

- ¿Qué clases presentan más errores?
- ¿Existen clases con vocabulario similar?
- ¿La matriz de confusión permite identificar diferencias entre Precision y Recall?

Paso 10: Analizar errores de clasificación

Inspeccionamos documentos cuya clase predicha es diferente de la clase verdadera.

```
errores = np.where(y_test != pred_tfidf)[0]

print("Numero de errores:", len(errores))

for i in errores[:5]:
    print("=" * 60)

    print("Texto:")
    print(X_test[i][:500])

    print("Clase real:")
    print(datos.target_names[y_test[i]])

    print("Clase predicha:")
    print(datos.target_names[pred_tfidf[i]])
```

¿Qué factores podrían explicar los errores?

- Textos demasiado cortos.
- Vocabulario compartido entre categorías.
- Documentos ambiguos.
- Información insuficiente.

Paso 11: Formular las hipótesis

Queremos investigar si TF-IDF proporciona un mejor F_1 macro que CountVectorizer.

Definimos:

$$A = \text{TF-IDF + LogisticRegression},$$

$$B = \text{CountVectorizer + LogisticRegression}.$$

La diferencia observada es:

$$\boxed{\delta(x) = F_1(A, x) - F_1(B, x).}$$

Hipótesis:

$$H_0 : \Delta \leq 0, \qquad H_1 : \Delta > 0,$$

donde Δ representa la diferencia verdadera (desconocida) de rendimiento entre ambos modelos en la población de interés.

Paired bootstrap test

Generaremos muestras bootstrap de los documentos del test set, conservando juntas las etiquetas verdaderas y las predicciones de ambos modelos.

Nivel de significancia: $\alpha = 0{,}05$.

Paso 12: Implementar el paired bootstrap

Generamos 10 000 muestras con reemplazo y calculamos la diferencia de F_1 macro para cada muestra.

```
rng = np.random.default_rng(42)

n = len(y_test)
B = 10000
clases = np.unique(y_train)

delta = f1_tfidf - f1_count
deltas_bootstrap = np.empty(B)

for i in range(B):
    idx = rng.choice(n, size=n, replace=True)

    f1_a = f1_score(y_test[idx], pred_tfidf[idx], labels=clases, average="macro", zero_division=0)
    f1_b = f1_score(y_test[idx], pred_count[idx], labels=clases, average="macro", zero_division=0)

    deltas_bootstrap[i] = f1_a - f1_b
```

¿Por qué es una prueba pareada?

Los mismos índices `idx` se utilizan para las etiquetas y las predicciones de ambos clasificadores.

Paso 13: Calcular el p-value

Siguiendo la aproximación del *p*-value:

$$p \approx \frac{1}{B} \sum_{i=1}^{B} \mathbb{I}\left(\delta(x^{(i)}) \geq 2\delta(x)\right).$$

```
p_value = np.mean(deltas_bootstrap >= 2 * delta)

print("Delta observado:", round(delta, 4))
print("p-value:", round(p_value, 4))

if delta > 0 and p_value < 0.05:
    print("Evidencia a favor de TF-IDF")
else:
    print("No se demuestra superioridad de TF-IDF")
```

Interpretación

Si $p < 0{,}05$ y la diferencia observada es positiva, rechazamos H_0 al nivel de significancia del 5 %.
Esto no significa que la probabilidad de que H_0 sea verdadera sea menor al 5 %.

Paso 14: Visualizar el paired bootstrap

Representamos la distribución de diferencias obtenidas mediante bootstrap.

```
plt.figure(figsize=(8, 4))

plt.hist(deltas_bootstrap, bins=40, edgecolor="black", alpha=0.75)

plt.axvline(delta, color="blue", linestyle="--", label="Delta observado")

plt.axvline(2*delta, color="red", linestyle="--", label="Umbral 2*delta")

plt.xlabel("Diferencia de F1 macro")
plt.ylabel("Frecuencia")
plt.legend()
plt.tight_layout()
plt.show()
```

Preguntas

¿Por qué la distribución bootstrap está aproximadamente centrada en $\delta(x)$? ¿Por qué comparamos con $2\delta(x)$ y no simplemente con $\delta(x)$?

Resumen

La evaluación de clasificadores de texto requiere:

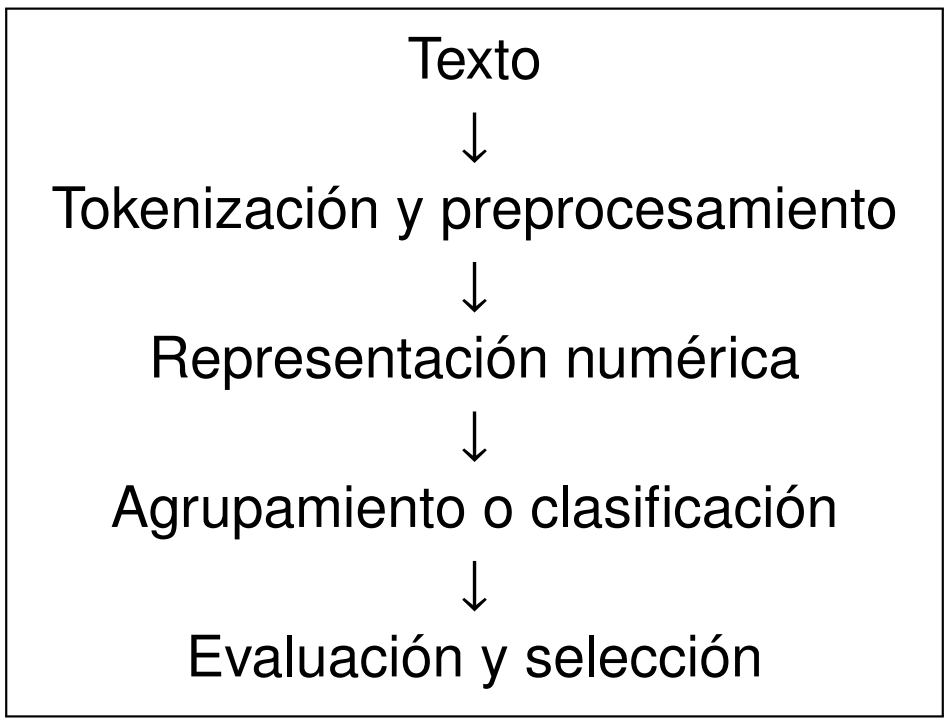
$$\boxed{\text{Métricas} + \text{Validación} + \text{Selección}}$$

En particular:

- Accuracy puede ser engañosa con clases desbalanceadas;
- Precision y Recall describen errores diferentes;
- F_1 combina ambas métricas;
- macro y micro averaging resumen problemas multiclase;
- cross-validation permite comparar configuraciones;
- el test set debe reservarse para la evaluación final;
- la regularización ayuda a controlar overfitting.

Tema 4: Aprendizaje de máquina para textos

La secuencia completa ha sido:



el texto se convierte en un problema estándar de aprendizaje de máquina.

¿Qué es un árbol de decisión?

- Modelo supervisado que puede resolver problemas de **clasificación** y **regresión**.
- Cada nodo interno contiene una condición, por ejemplo $x_j \leq t$.
- Cada rama representa el resultado de esa condición.
- Cada hoja proporciona una predicción.
- Puede capturar relaciones no lineales sin escalar ni centrar los atributos.

Ejemplo

¿La longitud del pétalo es menor o igual a 2,45 cm? Si sí, predecimos *setosa*; si no, hacemos otra pregunta.

Entrenar un árbol con Iris

Usaremos dos características de las flores y limitaremos la profundidad a dos niveles.

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier

iris = load_iris(as_frame=True)
X = iris.data[["petal length (cm)",
               "petal width (cm)"]].values
y = iris.target

arbol = DecisionTreeClassifier(
    max_depth=2, random_state=42
)
arbol.fit(X, y)
```

Visualizar el árbol entrenado

```
from sklearn.tree import export_graphviz

export_graphviz(
    arbol,
    out_file="iris_arbol.dot",
    feature_names=["longitud", "anchura"],
    class_names=iris.target_names,
    filled=True, rounded=True
)

from graphviz import Source

Source.from_file("iris_arbol.dot")
```

Observación

Cada nodo muestra una regla, el número de muestras, la distribución de clases y la clase predicha.

Cómo se realiza una predicción

1. Comenzamos en la **raíz**.
2. Evaluamos la condición del nodo.
3. Si se cumple, avanzamos por la rama izquierda; de lo contrario, por la derecha.
4. Repetimos hasta alcanzar una **hoja**.
5. La hoja determina la predicción.

Ejemplo Iris

- Longitud $\leq$ 2,45: *setosa*.
- Longitud $>$ 2,45 y anchura $\leq$ 1,75: *versicolor*.
- Longitud $>$ 2,45 y anchura $>$ 1,75: *virginica*.

Interpretación de los nodos

- **samples**: número de ejemplos de entrenamiento que llegan al nodo.
- **value**: cantidad de ejemplos de cada clase (o cantidades ponderadas).
- **class**: clase mayoritaria, usada para predecir.
- **gini**: medida de mezcla de clases dentro del nodo.

Pregunta

Si una hoja contiene 49 flores versicolor y 5 virginica, ¿qué clase predice? ¿Con qué confianza empírica?

Impureza de Gini

Para un nodo i con K clases, definimos

$$\boxed{G_i = 1 - \sum_{k=1}^{K} p_{i,k}^2}$$

donde $p_{i,k}$ es la proporción de ejemplos de clase k en el nodo i.

- Si todas las observaciones pertenecen a una clase, $G_i = 0$.
- Cuanto más mezcladas estén las clases, mayor es la impureza.

Ejemplo

Para proporciones $(0, 49/54, 5/54)$:

$$G = 1 - (49/54)^2 - (5/54)^2 \approx 0{,}168.$$

Ejercicio: calcular la impureza

Considera tres nodos con dos clases:

$$A : (10, 0), \qquad B : (5, 5), \qquad C : (8, 2).$$

1. Calcula G_A, G_B y G_C.
2. Ordena los nodos de menor a mayor impureza.
3. ¿Qué nodo preferirías como hoja de clasificación?

Ejercicio: calcular la impureza

Considera tres nodos con dos clases:

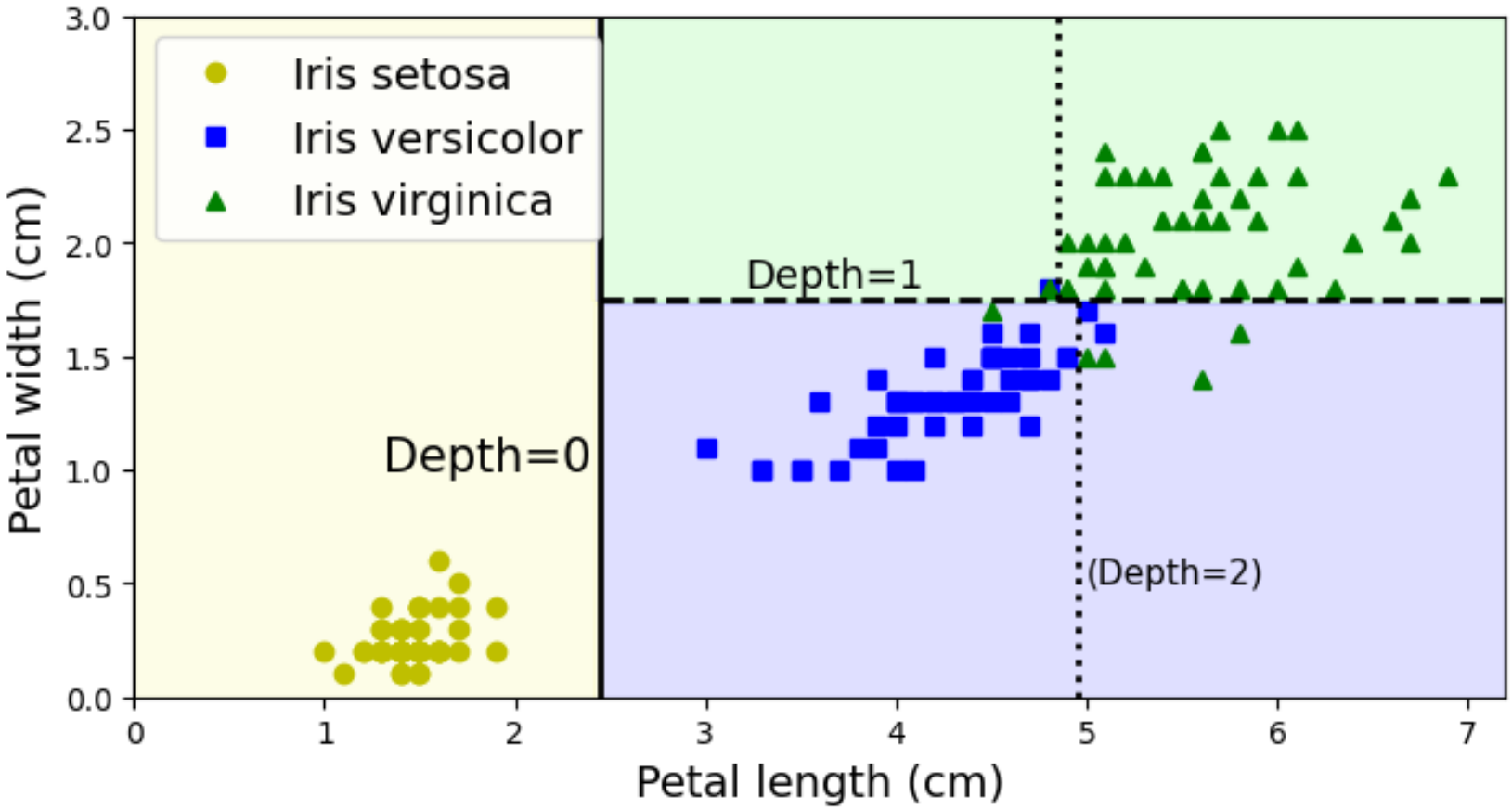
$$A : (10, 0), \qquad B : (5, 5), \qquad C : (8, 2).$$

1. Calcula G_A, G_B y G_C.
2. Ordena los nodos de menor a mayor impureza.
3. ¿Qué nodo preferirías como hoja de clasificación?

Resultados

$$G_A = 0, \qquad G_B = 0{,}5, \qquad G_C = 0{,}32.$$

Frontera de decisión del ejemplo:



Interpretabilidad: modelos de caja blanca y caja negra

Los modelos de aprendizaje automático pueden diferir en la facilidad con la que podemos explicar sus predicciones.

Modelos de caja blanca (White Box)

- Sus decisiones son fáciles de interpretar.
- Permiten identificar las reglas utilizadas para realizar una predicción.
- **Ejemplo:** árboles de decisión, cuyas reglas pueden seguirse manualmente.

Modelos de caja negra (Black Box)

- Pueden realizar predicciones muy precisas.
- Es difícil explicar de forma sencilla por qué producen una predicción específica.
- **Ejemplos:** Random Forest y redes neuronales.

Aprendizaje automático interpretable: busca desarrollar modelos cuyas decisiones puedan ser comprendidas por las personas.

¿Por qué es importante?

La interpretabilidad es especialmente relevante en medicina, finanzas, justicia y recursos humanos, donde es necesario comprender y evaluar las decisiones.

Probabilidades estimadas en las hojas

El árbol estima la probabilidad de clase usando la proporción de ejemplos de esa clase en la hoja:

$$\hat{P}(Y = k \mid X = x) = \frac{m_{hoja,k}}{m_{hoja}},$$

donde:

- m_{hoja}: número total de muestras que llegaron a esa hoja durante el entrenamiento.
- $m_{hoja,k}$: número de esas muestras que pertenecen a la clase k.
- k: clase para la cual queremos estimar la probabilidad.

```
print(arbol.predict_proba([[5, 1.5]]).round(3))
print(arbol.predict([[5, 1.5]]))
```

Resultado esperado: probabilidades $(0, 0{,}907, 0{,}093)$ y clase 1.

Limitación Todos los puntos que llegan a la misma hoja reciben exactamente las mismas probabilidades.

Algoritmo CART: idea general

CART: *Classification and Regression Trees*.

1. Examina posibles atributos k y umbrales t_k.
2. Divide los datos en dos grupos: $x_k \leq t_k$ y $x_k > t_k$.
3. Selecciona la división que minimiza una función de costo.
4. Aplica el procedimiento recursivamente en ambos grupos.
5. Se detiene al alcanzar límites de profundidad, tamaño o mejora.

Importante

Es un algoritmo **voraz**: selecciona la mejor división local, sin garantizar el árbol globalmente óptimo.

CART para clasificación

El costo de una división por atributo k y umbral t_k es

$$J(k, t_k) = \frac{m_{izq}}{m} G_{izq} + \frac{m_{der}}{m} G_{der}$$

- m: total de ejemplos en el nodo actual.
- m_{izq}, m_{der}: tamaños de los hijos.
- G_{izq}, G_{der}: impurezas de los hijos.

Interpretación

CART prefiere divisiones que produzcan grupos más puros, teniendo en cuenta cuántos ejemplos quedan en cada grupo.

Ejemplo: elegir una división

Un nodo contiene 20 observaciones. Consideramos dos divisiones:

- **División A**: 10 a la izquierda ($G = 0$) y 10 a la derecha ($G = 0{,}5$).
- **División B**: 5 a la izquierda ($G = 0{,}32$) y 15 a la derecha ($G = 0{,}32$).

$$J_A = \frac{10}{20}(0) + \frac{10}{20}(0{,}5) = 0{,}25$$

$$J_B = \frac{5}{20}(0{,}32) + \frac{15}{20}(0{,}32) = 0{,}32$$

Pregunta

¿Qué división elegiría CART y por qué?

Algoritmo CART: construcción del árbol

CART construye el árbol mediante un procedimiento **recursivo y greedy**.

1. Comienza con todos los ejemplos de entrenamiento en el nodo raíz.
2. Para cada característica k, considera diferentes umbrales t_k.
3. Evalúa la impureza ponderada de los nodos que produciría cada división:

$$J(k, t_k) = \frac{m_{izq}}{m} G_{izq} + \frac{m_{der}}{m} G_{der}.$$

4. Selecciona la característica y el umbral que minimizan la función de costo:

$$(k^*, t^*) = \arg \min_{k, t_k} J(k, t_k).$$

5. Divide los ejemplos en dos nodos hijos y repite el procedimiento en cada uno.

Criterio de parada

El proceso termina cuando se alcanza una condición de parada: nodo puro, profundidad máxima, número mínimo de muestras, etc.

CART: búsqueda de la mejor división

Para cada característica numérica, CART considera umbrales entre valores consecutivos ordenados.

Ejemplo: una característica representa las horas de estudio de seis estudiantes.

$$x_k = [1, 2, 3, 4, 5, 6]$$

Los umbrales candidatos son: $t_k \in \{1{,}5,\ 2{,}5,\ 3{,}5,\ 4{,}5,\ 5{,}5\}$.
Supongamos que las clases son: $y = [0, 0, 0, 1, 1, 1]$.

Umbral	**Distribución izquierda/derecha**	$\boldsymbol{J(k, t_k)}$
1,5	$(1, 0)/(2, 3)$	0,400
2,5	$(2, 0)/(1, 3)$	0,250
3,5	$(3, 0)/(0, 3)$	0,000
4,5	$(3, 1)/(0, 2)$	0,250
5,5	$(3, 2)/(0, 1)$	0,400

Selección

CART elige $t_k^* = 3{,}5$, porque produce dos nodos puros y minimiza $J(k, t_k)$.

CART es un algoritmo greedy

Un algoritmo **greedy** toma la mejor decisión disponible en cada paso, sin evaluar todas las consecuencias futuras.

En CART:

- En cada nodo, selecciona la división que minimiza la impureza ponderada.
- Una vez elegida, la división **no se reconsidera** durante el crecimiento del árbol.
- El algoritmo continúa recursivamente en los nodos hijos.
- No compara todas las posibles estructuras de árboles completos.

Ventaja: evita una búsqueda combinatoria de todas las estructuras posibles.
Limitación: puede producir árboles subóptimos y sensibles a pequeñas variaciones en los datos.

Complejidad computacional

Sea m el número de ejemplos y n el número de características.

- En un árbol aproximadamente balanceado, la profundidad es del orden de $\log_2 m$.
- **Predicción**: aproximadamente $O(\log m)$ comparaciones por instancia.
- **Entrenamiento**: aproximadamente $O(nm \log m)$ (por nodo) con búsqueda exhaustiva de las divisiones candidatas en el nodo.
- Limitar profundidad y atributos considerados puede acelerar el entrenamiento.

Entropía como criterio de impureza

Además de Gini, podemos utilizar la **entropía**:

$$H_i = -\sum_{k:p_{i,k}>0} p_{i,k} \log_2 p_{i,k}$$

- $H_i = 0$ cuando el nodo contiene una sola clase.
- Una mezcla equilibrada de clases aumenta la entropía.
- Gini y entropía suelen generar árboles similares.
- Gini suele ser algo más rápido de calcular.
- Cuando difieren, entropía genera árboles más balanceados.

Ejemplo

Para $(49/54, 5/54)$, $H \approx 0{,}445$ bits.

Comparar Gini y entropía en Python

```
arbol_gini = DecisionTreeClassifier(
    max_depth=3, criterion="gini", random_state=42
)
arbol_ent = DecisionTreeClassifier(
    max_depth=3, criterion="entropy", random_state=42
)

arbol_gini.fit(X, y)
arbol_ent.fit(X, y)

print(arbol_gini.get_depth(),
      arbol_gini.get_n_leaves())
print(arbol_ent.get_depth(),
      arbol_ent.get_n_leaves())
```

Visualizar los árboles:

```
export_graphviz(
    arbol_gini,
    out_file="arbol_gini.dot",
    feature_names=["longitud", "anchura"],
    class_names=iris.target_names,
    filled=True, rounded=True
)
Source.from_file("arbol_gini.dot")
```

```
export_graphviz(
    arbol_ent,
    out_file="arbol_ent.dot",
    feature_names=["longitud", "anchura"],
    class_names=iris.target_names,
    filled=True, rounded=True
)
Source.from_file("arbol_ent.dot")
```

Sobreajuste y regularización

Un árbol sin restricciones puede adaptarse demasiado a los ejemplos de entrenamiento.

- **Árbol profundo**: mayor flexibilidad, pero mayor riesgo de sobreajuste.
- **Árbol poco profundo**: menos varianza, pero puede subajustar.
- Regularizar significa limitar su capacidad de ajustar detalles del conjunto de entrenamiento.

Estrategia inicial

Reducir `max_depth` y aumentar `min_samples_leaf` suelen ser buenos puntos de partida.

Hiperparámetros de regularización

Hiperparámetro	Efecto
`max_depth`	Limita la profundidad máxima.
`min_samples_split`	Mínimo de muestras para dividir un nodo.
`min_samples_leaf`	Mínimo de muestras en cada hoja.
`max_leaf_nodes`	Limita el número de hojas.
`max_features`	Limita atributos evaluados en cada división.
`min_impurity_decrease`	Exige una mejora mínima de impureza.
`ccp_alpha`	Controla la *poda de mínima complejidad-costo* o MCCP (minimal cost-complexity pruning). Mayor `ccp_alpha`, más poda. Default = 0.

Regla práctica

Aumentar restricciones reduce la complejidad del árbol; demasiada regularización puede causar subajuste.

Poda por complejidad-costo (MCCP)

CART puede construir árboles muy profundos que **sobreajustan** los datos de entrenamiento.

Minimal Cost-Complexity Pruning (MCCP) simplifica el árbol eliminando ramas que aportan poca mejora respecto a su complejidad.

Función objetivo (penaliza el tamaño de árbol)

$$\boxed{R_\alpha(T) = R(T) + \alpha|T|}$$

- $R(T) = \sum_{\ell \in \text{hojas}(T)} \frac{m_\ell}{m} G_\ell$: impureza total ponderada de las hojas.
- $|T|$: número de hojas del árbol.
- $\alpha \geq 0$: penalización por complejidad.

- $\alpha = 0$: no se penaliza el tamaño.
- α grande: se favorecen árboles más pequeños.

MCCP: ¿qué ramas se eliminan?

MCCP utiliza una estrategia llamada **weakest-link pruning** (poda del eslabón más débil).

1. Se construye inicialmente un árbol grande.
2. Para cada nodo interno, se evalúa el costo de sustituir todo su subárbol por una hoja.
3. Se calcula:

$$\alpha_{eff}(t) = \frac{R(t) - R(T_t)}{|T_t| - 1}$$

4. Se poda el subárbol con menor α_{eff}.
5. Se repite el procedimiento para generar árboles progresivamente más pequeños.

Interpretación

α_{eff} mide el aumento de impureza por cada hoja eliminada.
Se eliminan primero las ramas cuya contribución es relativamente pequeña.

Ejemplo: regularización con `make_moons`

Próxima clase...

¡Gracias!